Na poprzednich zajęciach:

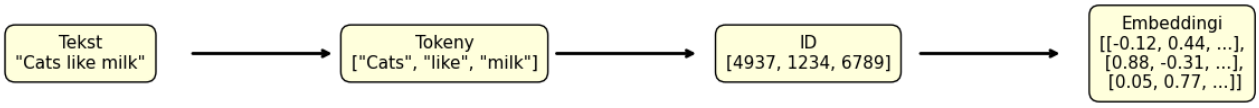
- **tokenizacja** – podział tekstu na słowa / tokeny,
- **spaCy Matcher** – znajdowanie wzorców na podstawie części mowy i cech morfologicznych.

Wektory słów

Tokenizacja a wektory słów

Tokenizacja -> Podziel tekst na kawałki (ID ze słownika)

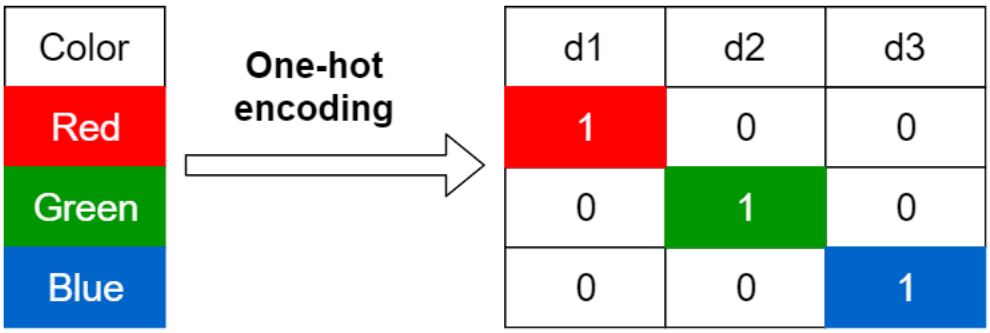
Embedding -> Przedstaw każde słowo w postaci liczb, aby algorytm mógł z nim dalej pracować



Komputer nie „rozumie" tekstu, tylko operuje na liczbach. Zatem każde słowo (np. „król", „kot", „miłość") trzeba jakoś zamienić na liczby, żeby można było nim manipulować w modelu.

Najprostszy pomysł — **one-hot encoding**

```
słownik = [kot, król, królowa]
"król" → [0, 1, 0]
```



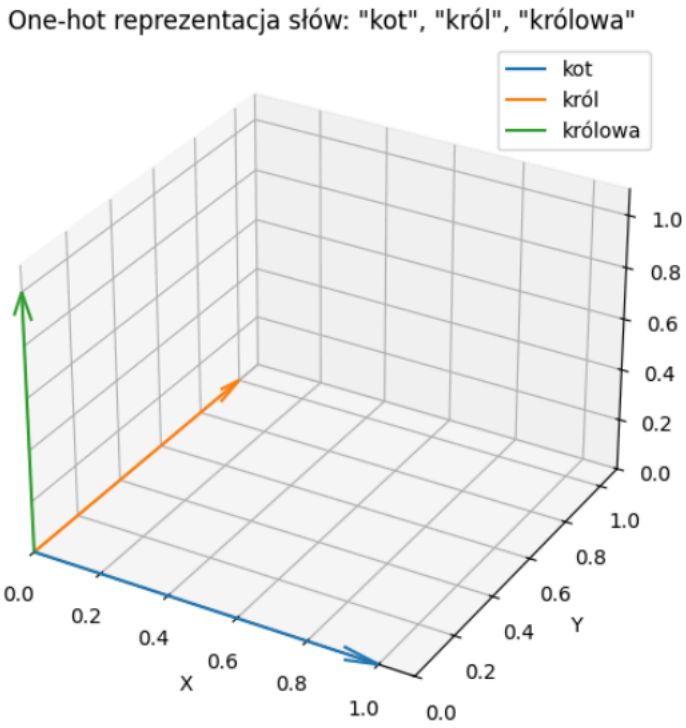
Dlaczego one-hot encoding jest problematyczny?

❌ Bardzo duży wymiar - Jeśli mamy słownik np. 100 000 słów, każdy wektor ma długość 100 000 — to ogromne marnotrawstwo pamięci.

❌ Brak informacji o znaczeniu - Wszystkie wektory są od siebie tak samo odległe — np. "kot" i "pies" są tak samo różne jak "kot" i "samolot". Nie ma żadnego pojęcia o podobieństwie semantycznym (znaczeniowym).

❌ Brak możliwości uogólniania - Model nie może „zrozumieć", że pewne słowa mają podobne znaczenie — każde słowo jest całkowicie niezależne.

```
słownik = [kot, król, królowa]
"kot" → [1, 0, 0]
"król" → [0, 1, 0]
"królowa" → [0, 0, 1]
```



Widzimy, że każdy wektor idzie w swoją stronę, każdy jest tak samo odległy od siebie (90 stopni). Żadne dwa nie są do siebie bliżej ani dalej — odległość między każdym z nich jest taka sama.

W naszym przykładzie: kot, król i królowa są tak samo podobni i różni = nie ma wartości semantycznej.

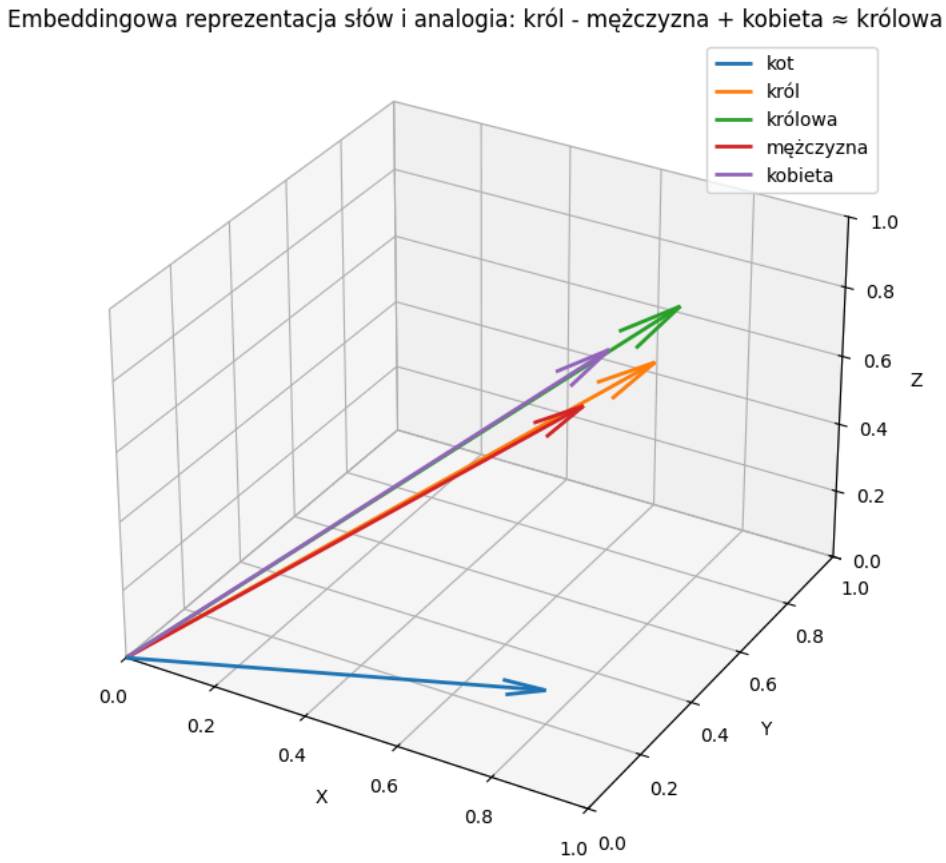
Embedingi

Embeddingi rozwiązują ten problem. W embeddingach uczymy model, żeby słowa, które występują w podobnych kontekstach, miały **podobne wektory**.

Dwa wektory są podobne, kiedy wskazują w podobnym kierunku w przestrzeni wielowymiarowej. Czyli: jeśli narysujesz te wektory jako strzałki zaczynające się w tym samym punkcie (np. w punkcie (0,0,0)) to mają **mały kąt między sobą**.

Przykład:

```
"king" → [0.25, 0.88, −0.12, 0.40, ...]
"queen" → [0.22, 0.90, −0.10, 0.45, ...]
"cat" → [−0.70, 0.11, 0.45, 0.20, ...]
```



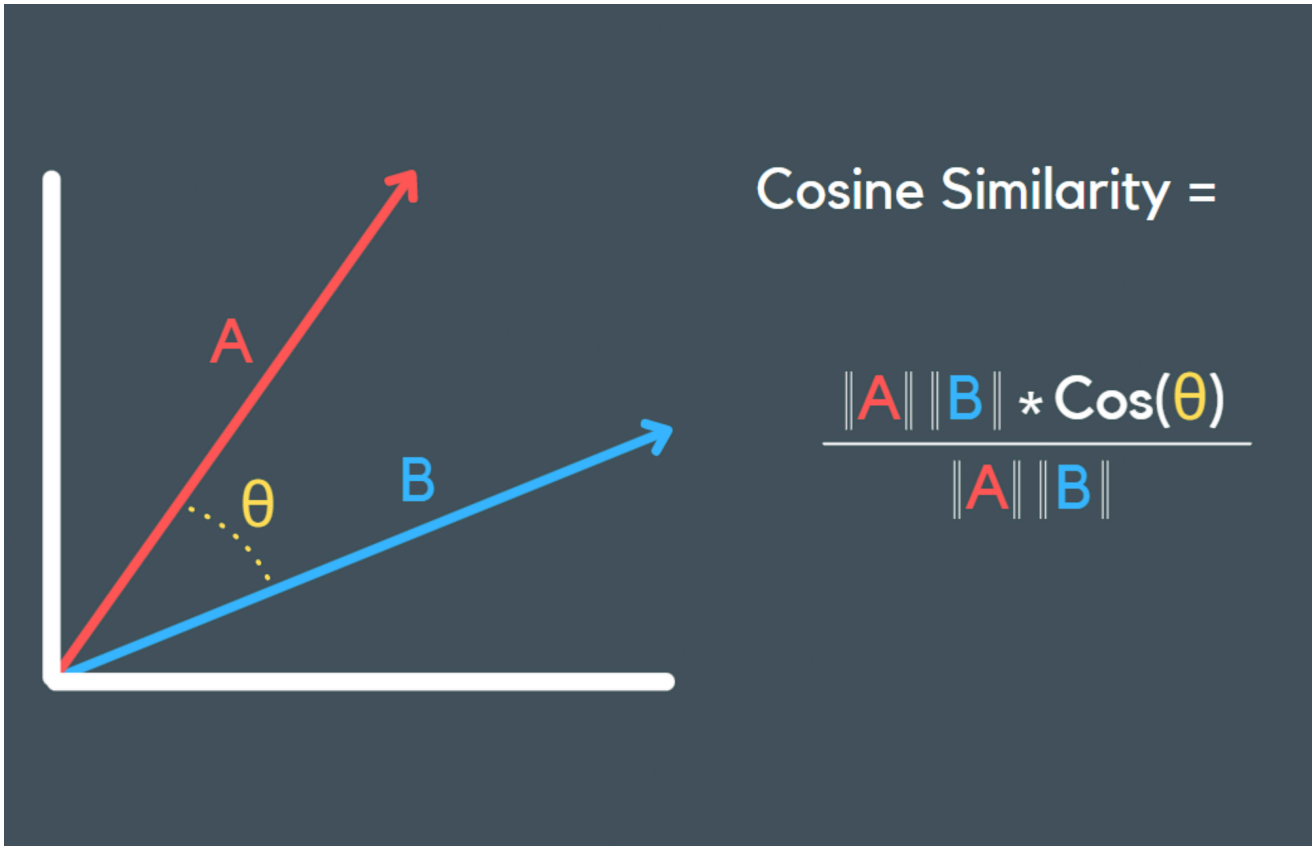

```
król − mężczyzna + kobieta ≈ królowa
```

Aby edytować zawartość komórki, kliknij ją dwukrotnie (lub naciśnij klawisz Enter)

```
# Sformatowano jako kod
```

Teraz punkty w przestrzeni układają się tak, że słowa o podobnym znaczeniu są blisko siebie. To jest właśnie **semantyczne podobieństwo** — podobieństwo w sensie znaczenia, a nie liter czy indeksów w tabeli.

Cosinusowe podobieństwo — jak mierzymy „bliskość znaczeń”



Cosinusowe podobieństwo to sposób na sprawdzenie, czy dwa wektory są „w tym samym kierunku":

- 1 ⟶ bardzo podobne (np. „kot" i „pies")
- 0 ⟶ brak podobieństwa (np. „kot" i „okno")
- -1 ⟶ całkowite przeciwieństwo

Ważne: Patrzymy nie na długość wektorów, tylko na kąt między nimi. Dwa wektory, które „patrzą" w tym samym kierunku, oznaczają słowa o podobnym znaczeniu.

Skąd się biorą te wektory?

Z treningu modelu językowego (np. Word2Vec, GloVe, FastText, BERT). Model uczy się na dużych korpusach tekstu, obserwując konteksty słów.

Efekt:

„king" i „queen" często występują w podobnych zdaniach (np. royal palace, throne, reign) ⟶ mają podobne wektory.

„dog" i „cat" też ⟶ inna grupa semantyczna.

Trochę praktyki

Zaczniemy od zapoznania się z biblioteką **gensim** Jej nazwa pochodzi od "GENerate SIMilar" — bo jej głównym celem jest znajdowanie podobieństwa między dokumentami lub słowami.

Użyjemy klasy **KeyedVectors** - to strukturą danych w gensim, która przechowuje wektory słów (embeddingi) wytrenowane dowolnym modelem. Czyli po prostu taki kontener do pracy z wektorami - niezależnie od tego, jak one zostały wytrenowane.

```
!pip install gensim
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.13/dist-packages (from gensim) (2.1.3)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.13/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.13/dist-packages (from gensim) (8.0.2)
Requirement already satisfied: wrapt in /usr/local/lib/python3.13/dist-packages (from smart_open>=1.8.1->gensim)
Downloading gensim-4.4.0-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.8/27.8 MB 63.7 MB/s eta 0:00:00
Installing collected packages: gensim
Successfully installed gensim-4.4.0
```

```
import gensim
from gensim.models import KeyedVectors
```

weźmiemy sobie polskie pre-trenowane wektory: http://dsmodels.nlp.ipipan.waw.pl pobieramy plik:

```
!wget http://dsmodels.nlp.ipipan.waw.pl/dsmodels/nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz --no-check-certi
```

```
--2026-10-12 19:17:21--  http://dsmodels.nlp.ipipan.waw.pl/dsmodels/nkjp+wiki-forms-restricted-300-skipg-ns.txt.g
Resolving dsmodels.nlp.ipipan.waw.pl (dsmodels.nlp.ipipan.waw.pl)... 213.135.36.94
Connecting to dsmodels.nlp.ipipan.waw.pl (dsmodels.nlp.ipipan.waw.pl)|213.135.36.94|:80... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://dsmodels.nlp.ipipan.waw.pl/dsmodels/nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz [following]
--2026-10-12 19:17:22--  https://dsmodels.nlp.ipipan.waw.pl/dsmodels/nkjp+wiki-forms-restricted-300-skipg-ns.txt.
Connecting to dsmodels.nlp.ipipan.waw.pl (dsmodels.nlp.ipipan.waw.pl)|213.135.36.94|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2152612134 (2.0G) [application/octet-stream]
Saving to: ‘nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz’

nkjp+wiki-forms-res 100%[===================>]   2.00G  11.4MB/s    in 2m 54s

2026-10-12 19:20:17 (11.8 MB/s) - ‘nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz’ saved [2152612134/2152612134]
```

Rozpakowujemy dane

```
!gzip -d "nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz"
```

Badamy format pliku

```
!head -n 100 nkjp+wiki-forms-restricted-300-skipg-ns.txt
#Pierwsza linia - lość słów (1982479), rozmiar wektora(300)
#Każda kolejna linia - słowo, liczby reperezentujące wektor
pracy -0.339222 0.225263 0.286258 0.264719 0.531696 0.053109 -0.272201 -0.205720 0.279595 0.215903 -0.055952 0.
były -0.102505 0.023722 -0.113885 0.347774 -0.238682 0.120603 -0.515847 -0.109189 0.041271 0.577869 -0.192093 -0
tam 0.321448 0.278772 0.074819 0.348238 0.167050 -0.159583 0.092537 -0.159371 -0.053519 0.518208 -0.075027 -0.1
osób -0.577304 -0.309684 0.332576 -0.130018 0.225140 0.141721 0.106455 0.383576 0.163807 0.312394 -0.101676 -0.
in -0.038397 0.109123 -0.062359 0.729193 0.147004 -0.184472 -0.049459 -0.168531 0.079803 0.369984 -0.042513 0.0
kiedy -0.165688 0.137390 -0.016459 -0.225001 -0.075221 0.031155 0.131225 -0.234085 0.040119 0.127815 -0.288528 -
miał -0.192369 0.167752 0.068614 0.873184 -0.038002 -0.354534 -0.153245 -0.218563 -0.147391 0.394977 -0.348807 -
```

```
pierwszy −0.266575 0.384896 0.660251 0.305221 −0.381303 0.212457 −0.357796 −0.329724 −0.171845 0.295718 −0.1113
jestem −0.028048 0.179036 −0.072213 −0.056222 0.007244 0.189300 0.034070 0.109402 −0.237497 −0.065510 −0.017094
czas −0.018453 0.137894 0.065990 0.001308 −0.044101 −0.204609 0.361818 −0.072674 0.096675 0.172855 −0.257486 −0
dnia −0.081396 0.056075 0.253912 0.308968 −0.199097 −0.115671 0.008291 0.250811 0.487077 0.468913 −0.318629 −0.
została −0.628995 −0.172253 −0.363241 0.600747 −0.374822 −0.125494 −0.159600 −0.193493 −0.106565 0.311173 −0.13
dziś −0.015517 −0.001961 −0.005589 0.377691 0.294157 −0.192328 0.066951 −0.038222 0.058811 0.372265 0.037670 0.
później 0.000400 0.341058 0.136597 0.256254 −0.182658 −0.218749 −0.318174 −0.047242 −0.102294 0.438479 −0.23758
```

(to może chwilę zająć)

```
#Ładowanie wektorów z pliku do naszego KeyedVectors
wv = KeyedVectors.load_word2vec_format('nkjp+wiki-forms-restricted-300-skipg-ns.txt')
```

```
wv.key_to_index.keys()
wv['dom']
```

```
array([-0.39294 , -0.156333,  0.017816, -0.208752,  0.442839, -0.389071,
       -0.22354 ,  0.119761,  0.201861,  0.136966,  0.005829, -0.036992,
       -0.075693, -0.424958,  0.031507, -0.214998, -0.11862 , -0.059799,
       -0.150971, -0.176567,  0.444136,  0.18925 ,  0.241265, -0.248733,
       -0.176206,  0.579075,  0.031083,  0.253453,  0.101754,  0.144021,
       -0.169662, -0.005122,  0.056225,  0.078038, -0.557848, -0.365922,
       -0.108863,  0.322074, -0.178232,  0.289982, -0.205699,  0.001991,
        0.001029, -0.074944,  0.173145,  0.044346, -0.032673, -0.060124,
        0.152098, -0.293121, -0.280878,  0.377744,  0.402627, -0.737609,
        0.020663, -0.01301 ,  0.134813, -0.052369,  0.046645, -0.024955,
        0.100392, -0.295437, -0.418354,  0.396864,  0.232666,  0.022819,
       -0.018474,  0.385309, -0.084371,  0.269105,  0.149824, -0.161652,
        0.282115,  0.177849, -0.241524, -0.240634, -0.111456,  0.070668,
       -0.216499, -0.139156, -0.576622, -0.136423,  0.403575,  0.155968,
        0.218948, -0.002413,  0.123077,  0.102393,  0.741444,  0.390994,
       -0.110964,  0.134329,  0.265171,  0.251443, -0.221384, -0.344603,
        0.283788,  0.284915,  0.057016,  0.384028, -0.733822,  0.303532,
       -0.583847, -0.02494 , -0.218198,  0.018208,  0.145667,  0.255729,
        0.498478, -0.112536, -0.047852, -0.337887,  0.031235, -0.432209,
        0.208977,  0.012221,  0.19098 , -0.206233, -0.113016,  0.111511,
       -0.059505,  0.027593, -0.100111, -0.230436, -0.078137,  0.278976,
        0.19794 ,  0.210459, -0.155359, -0.112397,  0.26439 , -0.199999,
        0.058368,  0.032107,  0.110054,  0.051876, -0.240713,  0.277577,
        0.289985, -0.200094, -0.101406,  0.131032, -0.059501,  0.134268,
        0.148928, -0.085373,  0.030669,  0.068627, -0.215063, -0.081981,
       -0.052877, -0.291941, -0.458875,  0.050618,  0.214273,  0.184849,
       -0.038208,  0.036236, -0.642125, -0.464941,  0.372085, -0.187011,
       -0.069729,  0.035051,  0.099067,  0.057524,  0.006579,  0.533439,
       -0.198236, -0.344194, -0.067796, -0.063289, -0.069952, -0.041706,
        0.142563,  0.05342 , -0.054871,  0.2577  ,  0.50673 ,  0.208496,
        0.650207, -0.252643,  0.320542, -0.54734 , -0.637634, -0.51038 ,
        0.025401,  0.05803 ,  0.16656 ,  0.257388,  0.305958, -0.219199,
        0.2086  , -0.251887, -0.185647, -0.027838, -0.186104, -0.070295,
       -0.094713,  0.102284, -0.123802,  0.083343,  0.315851, -0.031123,
        0.238093,  0.149717, -0.004316, -0.181932, -0.086956,  0.531083,
        0.112422,  0.231151,  0.242227,  0.012977, -0.095725, -0.283862,
       -0.061226,  0.311026,  0.146489,  0.054716, -0.048658, -0.282848,
       -0.054836, -0.030167, -0.090273, -0.435833, -0.287074, -0.351567,
       -0.002671, -0.020709, -0.276286,  0.281762, -0.114932, -0.243566,
       -0.188988,  0.007132,  0.146958, -0.269132, -0.034678,  0.164219,
       -0.090405,  0.044055, -0.405617,  0.162068,  0.008641,  0.142725,
       -0.065804,  0.136329,  0.247081, -0.144136, -0.007592,  0.113807,
        0.14899 , -0.012305, -0.346247,  0.111213,  0.011855,  0.250259,
        0.068319,  0.100238,  0.010446,  0.127471, -0.033004, -0.079141,
       -0.531317,  0.318433,  0.562701,  0.354442,  0.001789,  0.171425,
        0.010197,  0.043681, -0.031715,  0.505568, -0.453908,  0.050882,
       -0.294158, -0.279005, -0.121981,  0.182517,  0.419662,  0.325643,
       -0.027132, -0.020383,  0.133923, -0.047128,  0.406531, -0.385169,
       -0.148565, -0.109004, -0.062878, -0.236207,  0.008543,  0.044215,
        0.216761,  0.017197, -0.034889,  0.311272,  0.130243, -0.122846],
      dtype=float32)
```

Znajdziemy najbardziej podobne słowo do słowa kobieta.

```
wv.most_similar('kobieta')
```

```
[('mężczyzna', 0.7043285369873047),
 ('dziewczyna', 0.6813511252403259),
 ('ciężarna', 0.612939715385437),
 ('dziewczynka', 0.6094770431518555),
 ('50-letnia', 0.5805984139442444),
 ('młoda', 0.5788260102272034),
 ('40-letnia', 0.5631040930747986),
 ('kobietą', 0.5611501336097717),
 ('dama', 0.5526077747344971),
 ('mężczyzną', 0.5519220232963562)]
```

Znajdź najbardziej podobne słowo do słowa "kobieta", które nie jest podobne do słowa "mężczyzna"

```
wv.most_similar(positive=['kobieta'], negative=['mężczyzna'])
```

```
[('Teresa', 0.3597542345523834),
 ('Irlandka', 0.35389503836631775),
 ('Barbara', 0.33941537141799927),
 ('Helena', 0.33837634325027466),
 ('Eleonora', 0.32829549908638),
 ('mianowana', 0.3252837657928467),
 ('Maryse', 0.32269802689552307),
 ('Emmanuella', 0.32241636514663696),
 ('czarnoskóra', 0.3224010169506073),
 ('Izabela', 0.3222866654396057)]
```

W grupie "pies kot żubr budyń" znajdź słowo, które nie pasuje.

```
wv.doesnt_match(['pies', 'kot', 'żubr', 'budyń'])
```

```
'budyń'
```

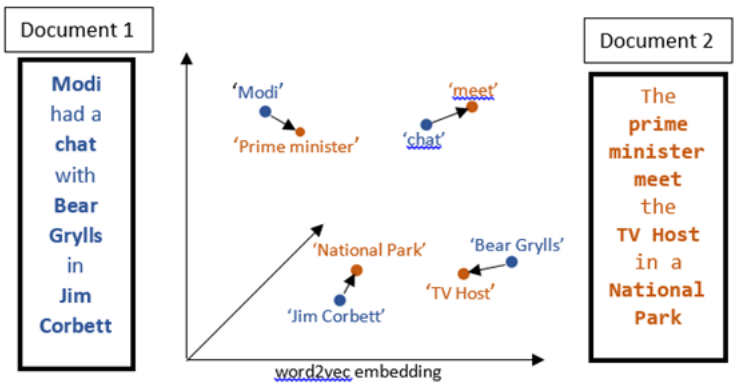
zbadaj podobieństwo między kotem a psem

```
wv.similarity('kot', 'pies')
```

```
np.float32(0.6970217)
```

Teraz zajmiemy się podobieństwami pomiędzy zdaniami

Word mover distance to minimalna suma odległości, jaką trzeba „pokonać", aby słowa z jednego zdania pokryły znaczeniowo słowa z drugiego zdania.




```
!pip install POT #Po tej instalacji gensim może szybciej i dokładniej liczyć wmdistance
```

```
Collecting POT
  Downloading pot-0.9.7.post1-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (44 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.2/44.2 kB 1.7 MB/s eta 0:00:00
Requirement already satisfied: numpy>=1.16 in /usr/local/lib/python3.13/dist-packages (from POT) (2.1.3)
Requirement already satisfied: scipy>=1.6 in /usr/local/lib/python3.13/dist-packages (from POT) (1.16.3)
Downloading pot-0.9.7.post1-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (32.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 32.9/32.9 MB 17.7 MB/s eta 0:00:00
Installing collected packages: POT
Successfully installed POT-0.9.7.post1
```

```
sentence_1 = 'Byłem dzisiaj w kinie'.lower().split()
sentence_2 = 'Poszłam wczoraj do teatru'.lower().split()
```

```
wv.wmdistance(sentence_1, sentence_2)
```

```
1.176677407878612
```

WMD nie jest ograniczone do 0-1, ale przy normalizacji embeddingów zwykle wartości mieszczą się w 0-4 dla krótkich zdań.

wynik ~1.25 sugeruje, że zdania są umiarkowanie podobne semantycznie.

Dla porównania:

- Dwa identyczne zdania → WMD ≈ 0
- Zupełnie różne zdania → WMD > 2.5-3

Flair

Flair to biblioteka NLP od Zalando Research (Python)

Główna idea:

- Klasyczne embeddingi (Word2Vec, GloVe, FastText) dają stały wektor dla każdego słowa, niezależnie od kontekstu:

  "bank" w *"river bank"* i *"money bank"* → ten sam wektor.

- Flair tworzy kontekstowe embeddingi(wektor słowa zależy od jego zdania / kontekstu)

  więc "*bank*" w "*river bank*" będzie inny niż w "*money bank*".

Flair uczy się tego przez modele LSTM działające w dwóch kierunkach (forward i backward).

```
!pip install flair
```

```
Collecting ftfy>=6.1.0 (from flair)
  Downloading ftfy-6.3.1-py3-none-any.whl.metadata (7.3 kB)
Requirement already satisfied: gdown>=4.4.0 in /usr/local/lib/python3.13/dist-packages (from flair) (5.2.2)
Requirement already satisfied: huggingface-hub>=0.10.0 in /usr/local/lib/python3.13/dist-packages (from flair)
Collecting langdetect>=1.0.9 (from flair)
  Downloading langdetect-1.0.9.tar.gz (981 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 981.5/981.5 kB 19.5 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: lxml>=4.8.0 in /usr/local/lib/python3.13/dist-packages (from flair) (6.1.3)
Requirement already satisfied: matplotlib>=2.2.3 in /usr/local/lib/python3.13/dist-packages (from flair) (3.10.
Requirement already satisfied: more-itertools>=8.13.0 in /usr/local/lib/python3.13/dist-packages (from flair) (
Collecting mpld3>=0.3 (from flair)
  Downloading mpld3-0.5.12-py3-none-any.whl.metadata (5.3 kB)
Collecting pptree>=3.1 (from flair)
  Downloading pptree-3.1.tar.gz (3.0 kB)
  Preparing metadata (setup.py) ... done
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.13/dist-packages (from flair) (
Collecting pytorch-revgrad>=0.2.0 (from flair)
  Downloading pytorch_revgrad-0.2.0-py3-none-any.whl.metadata (1.7 kB)
Requirement already satisfied: regex>=2022.1.18 in /usr/local/lib/python3.13/dist-packages (from flair) (2025.1
Requirement already satisfied: scikit-learn>=1.0.2 in /usr/local/lib/python3.13/dist-packages (from flair) (1.6
Collecting segtok>=1.5.11 (from flair)
  Downloading segtok-1.5.11-py3-none-any.whl.metadata (9.0 kB)
Collecting sqlitedict>=2.0.0 (from flair)
  Downloading sqlitedict-2.1.0.tar.gz (21 kB)
  Preparing metadata (setup.py) ... done
Requirement already satisfied: tabulate>=0.8.10 in /usr/local/lib/python3.13/dist-packages (from flair) (0.9.0)
Requirement already satisfied: torch>=1.13.1 in /usr/local/lib/python3.13/dist-packages (from flair) (2.11.0+cpu
Requirement already satisfied: tqdm>=4.63.0 in /usr/local/lib/python3.13/dist-packages (from flair) (4.67.3)
Collecting transformer-smaller-training-vocab>=0.2.3 (from flair)
  Downloading transformer_smaller_training_vocab-0.4.2-py3-none-any.whl.metadata (7.9 kB)
Collecting transformers<5.0.0,>=4.25.0 (from transformers[sentencepiece]<5.0.0,>=4.25.0->flair)
  Downloading transformers-4.57.6-py3-none-any.whl.metadata (43 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.0/44.0 kB 2.7 MB/s eta 0:00:00
Collecting wikipedia-api>=0.5.7 (from flair)
  Downloading wikipedia_api-0.16.0-py3-none-any.whl.metadata (44 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.7/44.7 kB 2.9 MB/s eta 0:00:00
Collecting bioc<3.0.0,>=2.0.0 (from flair)
  Downloading bioc-2.1-py3-none-any.whl.metadata (4.6 kB)
Collecting jsonlines>=1.2.0 (from bioc<3.0.0,>=2.0.0->flair)
  Downloading jsonlines-4.0.0-py3-none-any.whl.metadata (1.6 kB)
Collecting intervaltree (from bioc<3.0.0,>=2.0.0->flair)
  Downloading intervaltree-3.2.1-py2.py3-none-any.whl.metadata (12 kB)
Collecting docopt (from bioc<3.0.0,>=2.0.0->flair)
  Downloading docopt-0.6.2.tar.gz (25 kB)
  Preparing metadata (setup.py) ... done
Collecting botocore<1.44.0,>=1.43.112 (from boto3>=1.20.27->flair)
  Downloading botocore-1.43.112-py3-none-any.whl.metadata (5.6 kB)
Collecting jmespath<2.0.0,>=0.7.1 (from boto3>=1.20.27->flair)
  Downloading jmespath-1.1.0-py3-none-any.whl.metadata (7.6 kB)
Collecting s3transfer<0.20.0,>=0.19.0 (from boto3>=1.20.27->flair)
  Downloading s3transfer-0.19.2-py3-none-any.whl.metadata (1.7 kB)
Requirement already satisfied: wrapt<3,>=1.16 in /usr/local/lib/python3.13/dist-packages (from deprecated>=1.2.
Requirement already satisfied: wcwidth in /usr/local/lib/python3.13/dist-packages (from ftfy>=6.1.0->flair) (0.
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.13/dist-packages (from gdown>=4.4.0->fl
Requirement already satisfied: filelock in /usr/local/lib/python3.13/dist-packages (from gdown>=4.4.0->flair) (
Requirement already satisfied: requests[socks] in /usr/local/lib/python3.13/dist-packages (from gdown>=4.4.0->f
Requirement already satisfied: click<9.0.0,>=8.4.2 in /usr/local/lib/python3.13/dist-packages (from huggingface-
```

```
from flair.embeddings import FlairEmbeddings, StackedEmbeddings
from flair.data import Sentence
```

```
WARNING:torchao.kernel.intmm:Warning: Detected no triton, on systems without Triton certain kernels will not work
```

```
embeddings = FlairEmbeddings('news-forward')
```

```
#tworzymy objekty Sentence
s1 = Sentence("How does it feel to be a scribe?")
s2 = Sentence("There is no such thing as simply good or bad")
sentences = [s1, s2]


# Obliczamy embeddingi(zanurzamy słowa)
embeddings.embed(sentences)

for sentence in sentences: #Zanurzone zdania nadal są obiektem Sentence
  print(sentence)
```

```
2026-10-12 19:55:29,768 https://flair.informatik.hu-berlin.de/resources/embeddings/flair/news-forward-0.4.1.pt no
100%|██████████| 69.7M/69.7M [00:05<00:00, 14.5MB/s]2026-10-12 19:55:35,308 copying /tmp/tmpabmy4hyq to cache at

2026-10-12 19:55:35,560 removing temp file /tmp/tmpabmy4hyq
Sentence[9]: "How does it feel to be a scribe?"
Sentence[10]: "There is no such thing as simply good or bad"
```

```
#dostajemy embeddingi poszczególnych słów
for sentence in sentences:
  for token in sentence:
    print(token.embedding)
#embeddingi są tensorami
```

```
tensor([-0.0011, -0.0021,  0.0038,  ..., -0.0004,  0.0099,  0.0547])
tensor([-0.0016, -0.0260,  0.0388,  ...,  0.0031, -0.0070,  0.0025])
tensor([0.0174, 0.0181, 0.0148,  ..., 0.0051, 0.0051, 0.0130])
tensor([ 0.0001,  0.0128,  0.0476,  ..., -0.0084, -0.0023,  0.0106])
tensor([-2.1672e-03,  8.3753e-05,  2.5102e-02,  ..., -2.8347e-04,
        -1.2090e-02,  2.9513e-02])
tensor([-0.0032,  0.0020,  0.0280,  ...,  0.0004, -0.0017, -0.0106])
tensor([ 5.3283e-03,  3.8049e-05, -2.2042e-02,  ..., -4.1646e-04,
         1.9279e-02, -1.2351e-02])
tensor([-1.0594e-03,  4.8670e-05,  2.1462e-02,  ..., -1.1294e-05,
        -4.3308e-03,  3.1280e-03])
tensor([ 8.7317e-04,  3.7862e-06, -4.6351e-04,  ..., -1.4222e-04,
         5.4708e-03,  3.0888e-03])
tensor([-0.0012, -0.0002, -0.1499,  ...,  0.0024,  0.0017,  0.1648])
tensor([ 0.0078, -0.0047,  0.0564,  ..., -0.0012,  0.0325,  0.0035])
tensor([ 5.8074e-02, -7.9190e-05,  9.8895e-02,  ..., -2.7518e-03,
         8.8326e-04,  2.6957e-02])
tensor([-0.0004, -0.0036,  0.0781,  ..., -0.0004, -0.0344, -0.0005])
tensor([ 0.0014,  0.0033, -0.0118,  ..., -0.0005, -0.0164,  0.0293])
tensor([ 1.2536e-02, -1.0569e-03, -1.1876e-02,  ..., -2.9878e-05,
         4.0988e-03,  2.8744e-02])
tensor([-0.0053, -0.0011,  0.0058,  ..., -0.0018,  0.0212,  0.0082])
tensor([ 2.0060e-04,  9.6346e-04, -2.2168e-02,  ...,  6.8233e-05,
        -1.3982e-01,  3.5051e-03])
tensor([-9.0032e-05, -1.7683e-03,  1.0521e-02,  ..., -1.9229e-03,
        -1.2467e-05,  3.2030e-02])
tensor([-0.0006,  0.0004, -0.0019,  ..., -0.0005,  0.0061, -0.0073])
```

```
import numpy as np
```

```
for sentence in sentences:
  for token in sentence:
    print(np.array(token.embedding))
```

```
[-0.00108573 -0.00213397  0.00377833 ... -0.00035715  0.0099433
  0.05467612]
[-0.00163258 -0.02604993  0.03880594 ...  0.00308893 -0.00699765
  0.00248321]
[0.01740974 0.01807804 0.0147752  ... 0.00507008 0.00508688 0.01302868]
[ 0.00013419  0.01282798  0.04760626 ... -0.00837356 -0.00231261
  0.01057426]
[-2.1671702e-03  8.3752588e-05  2.5102206e-02 ... -2.8347093e-04
 -1.2090425e-02  2.9513163e-02]
[-0.00319913  0.00195263  0.02804686 ...  0.00040722 -0.00174229
 -0.01062463]
[ 5.32832043e-03  3.80494457e-05 -2.20420863e-02 ... -4.16457391e-04
  1.92785766e-02 -1.23509625e-02]
[-1.0594028e-03  4.8669703e-05  2.1461956e-02 ... -1.1294257e-05
 -4.3307757e-03  3.1279572e-03]
[ 8.7317190e-04  3.7861769e-06 -4.6351264e-04 ... -1.4222013e-04
  5.4707765e-03  3.0888412e-03]
[-0.00118427 -0.00018427 -0.14988413 ...  0.00244023  0.00166776
  0.16482687]
[ 0.00780449 -0.00474085  0.056434   ... -0.00120921  0.03250852
  0.00350805]
[ 5.8073573e-02 -7.9189987e-05  9.8894574e-02 ... -2.7517818e-03
  8.8326377e-04  2.6957348e-02]
[-0.0003961  -0.00361209  0.07808607 ... -0.00039112 -0.03444917
 -0.00053953]
[ 0.00144947  0.00326684 -0.01180778 ... -0.00050383 -0.01641377
```

```
  0.02928468]
[ 1.2535800e-02 -1.0568546e-03 -1.1875577e-02 ... -2.9878282e-05
  4.0988261e-03  2.8744373e-02]
[-0.00529048 -0.00108928  0.00580226 ... -0.00176485  0.02117513
  0.00820003]
[ 2.0059671e-04  9.6345815e-04 -2.2167904e-02 ...  6.8233297e-05
 -1.3981722e-01  3.5051273e-03]
[-9.00320447e-05 -1.76826888e-03  1.05207525e-02 ... -1.92288996e-03
 -1.24673452e-05  3.20298225e-02]
[-0.00058445  0.00037377 -0.00193683 ... -0.00046098  0.00614759
 -0.00726028]
/tmp/ipykernel_915/932037997.py:3: DeprecationWarning: __array__ implementation doesn't accept a copy keyword, so
  print(np.array(token.embedding))
```

Embeddingi dla zdań

```
from flair.embeddings import DocumentPoolEmbeddings
```

```
sentence_embeddings = DocumentPoolEmbeddings([FlairEmbeddings('news-forward')])
```

```
sentence_embeddings.embed(sentences)
```

```
for sentence in sentences:
  print(sentence.embedding)
```

```
tensor([ 0.0016,  0.0005,  0.0175,  ..., -0.0001,  0.0014,  0.0104])
tensor([ 0.0073, -0.0008,  0.0052,  ..., -0.0007, -0.0124,  0.0289])
```

Zadania

Zadanie1

Załaduj polski model kontekstowych embeddingów Flair:

```
embeddings = FlairEmbeddings('pl-forward')
```

```
2026-10-12 19:56:30,040 https://flair.informatik.hu-berlin.de/resources/embeddings/flair/lm-polish-forward-v0.2.p
100%|██████████| 80.3M/80.3M [00:06<00:00, 13.8MB/s]2026-10-12 19:56:37,167 copying /tmp/tmpibo7n0ix to cache at

2026-10-12 19:56:37,412 removing temp file /tmp/tmpibo7n0ix
```

```
s1 = Sentence("Zamek w Malborku jest bardzo stary.")
s2 = Sentence("Suwak w kurtce to zamek błyskawiczny")
s3 = Sentence("Sprawdź, czy zamek w drzwiach działa")
sentences = [s1, s2, s3]
```

Oblicz embeddingi dla wszystkich zdań:

```
embeddings.embed(sentences)
```

```
[Sentence[7]: "Zamek w Malborku jest bardzo stary.",
 Sentence[6]: "Suwak w kurtce to zamek błyskawiczny",
 Sentence[7]: "Sprawdź, czy zamek w drzwiach działa"]
```

Wypisz każde zdanie i dla słowa "zamek", wypisz pierwsze 5 wymiarów embeddingu:

```
for sentence in sentences:
  print(sentence)
  for token in sentence:
    if token.text.lower() == "zamek":
      print(token.embedding[:5])
```

```
Sentence[7]: "Zamek w Malborku jest bardzo stary."
tensor([-5.4509e-03, -2.1590e-02,  4.5458e-05, -1.7073e-04, -4.3133e-04])
Sentence[6]: "Suwak w kurtce to zamek błyskawiczny"
tensor([-0.0043, -0.0131,  0.0004, -0.0001, -0.0004])
Sentence[7]: "Sprawdź, czy zamek w drzwiach działa"
tensor([-4.3021e-03, -2.1082e-02,  7.7413e-04, -6.9931e-05, -3.6798e-04])
```

Czy embeddingi są różne w zależności od znaczenia słowa?

Odpowiedź: Tak

Zadanie 2

Stwórz bota z bazą przykładownych pytań i odpowiedzi, zwracającego odpowiedź na zapytanie na podstawie podobieństwa (WMD albo flair document embeddings) pomiędzy zapytniem użytownika a pytaniami w swojej bazie

Przykład:

Zbiór pytań i odpowiedzi :

```
[
  {"q1" : "a1"},
  {"q2" : "a2"},
  {"q3" : "a3"},
  ...
  {"qn" : "an"},
]
```

Wektoryzujemy pytania:

```
  {"q1_vecotorized" : "a1"},
  {"q2_vecotorized" : "a2"},
  {"q3_vecotorized" : "a3"},
  ...
  {"qn_vecotorized" : "an"},
```

Dla pytania użytkownia pytania q_user wektoryzujemy je i liczmy podobieństwo z pytaniami w bazie

```
sim("q_user_vectorized","q1_vecotorized"), sim("q_user_vectorized","q2_vecotorized"), ... ("q_user_vectorized""qn_v
```

Zwracamy odpowiedź do pytania z bazy najbardziej podobnego do pytania użytkownika

$$argmax(sim(q_{user}vectorized, q_i vecotorized))$$

Do liczenia podobnieństma można użyć np https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.distance.cosine.html ale wtedy robimy argmin bo

$$cosine\ distance = 1 - cosine\ similarity$$

```
qa = [
    {"q": "Gdzie znajduje się Warszawa?", "a": "Warszawa leży w Polsce."},
    {"q": "Czym jest komputer?", "a": "Komputer to urządzenie elektroniczne."},
    {"q": "Co jedzą myszy?", "a": "Myszy jedzą między innymi ziarna i nasiona."}
]
```

```
from flair.data import Sentence
from flair.embeddings import DocumentPoolEmbeddings
from scipy.spatial.distance import cosine
import numpy as np

sentence_embeddings = DocumentPoolEmbeddings([embeddings])

def calculate_similarity(vec1, vec2):
    if len(vec1) == 0 or len(vec2) == 0:
        return -1
    return 1 - cosine(vec1, vec2)

def bot(user_question):
    user_sentence = Sentence(user_question)
    sentence_embeddings.embed(user_sentence)

    best_similarity = -1
    best_question = ""
    best_answer = ""

    for item in qa:
```